PR Wrangler: problem, solution, tools and skills

A detailed write-up of what PR Wrangler is, the problem it solves, how it solves it, and every tool and skill that went into it.

---

1. The problem

AI coding tools made writing code cheap. Reviewing it didn't get any cheaper.

Picture a team of seven developers who each use a coding agent. Each of them can open several pull requests a day. The tech lead opens GitHub in the morning and finds 15 open PRs, with no idea:

- **which ones are safe** to merge in a minute and which need a real review;
- **which ones hide a problem**, like a "restyle" PR that quietly deletes a safety check, or a one-line filter that's inverted;
- **which ones add risky dependencies**: a vulnerable package, a duplicate of a library the app already uses, or two PRs pinning different versions of the same package;
- **which ones collide**: two PRs that rewrite the same lines, so one has to be merged first and the other redone;
- **which ones are blocked**, because they're built on top of another open PR.

GitHub doesn't help much here. It compares each PR to `main` on its own, so it marks all of them "mergeable" even when they conflict with each other.

The bottleneck has moved from writing code to deciding what deserves human attention first.

---

2. The solution in one paragraph

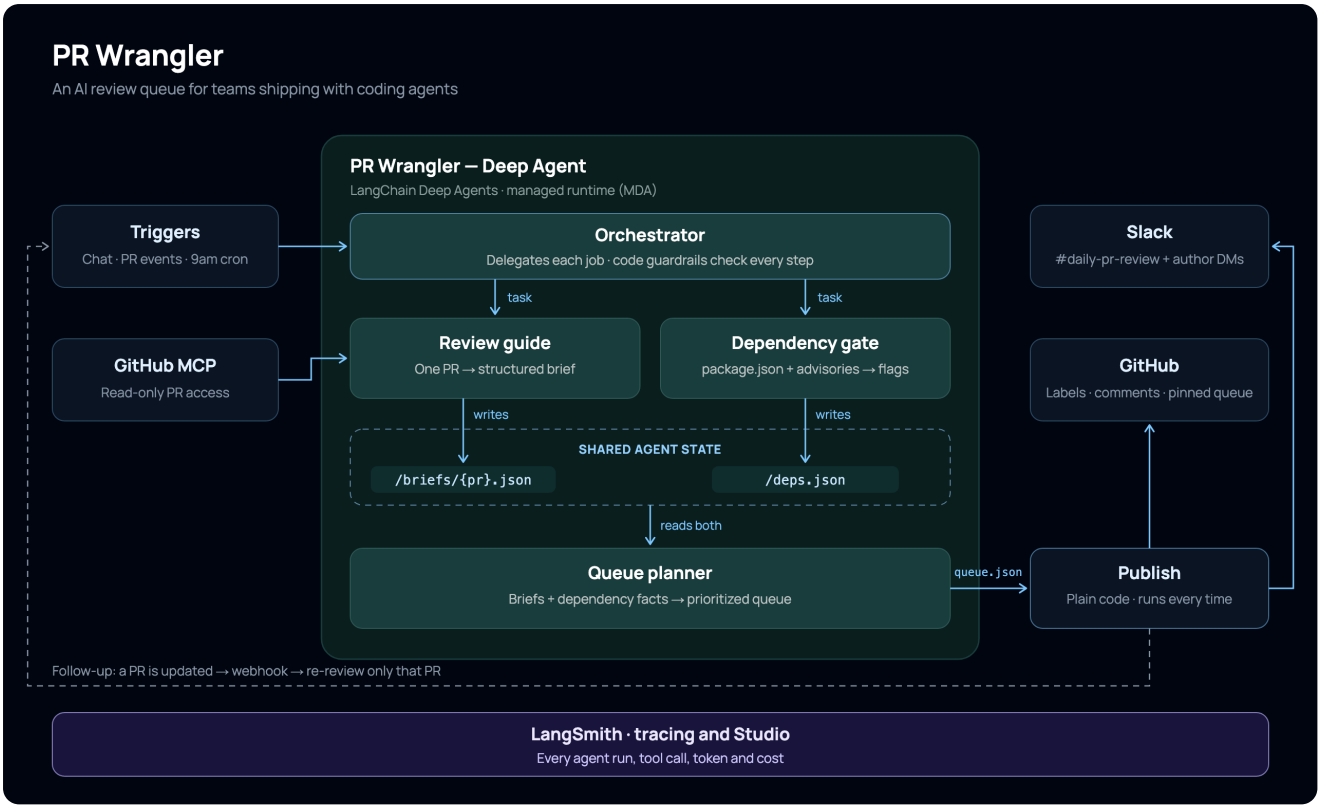
PR Wrangler is a multi-agent system that reads every open PR and turns the pile into an **ordered review queue**. One **orchestrator** agent delegates to three specialist **subagents**:

- a **review guide** that reads one PR in depth;
- a **dependency gate** that checks every package change;
- a **queue planner** that ranks everything into lanes.

The results are written back where the team already works:

- **GitHub:** labels, a brief on each PR, a "Start here" comment on the exact line, and a pinned Review Queue issue.
- **Slack:** a channel digest, plus a DM to each author whose PR needs a fix.

It never approves or merges. It writes the first draft of the review, and a human decides.



3. How it works, end to end

Step	Who	What happens	Output
1	Trigger	Someone types "Wrangle the queue." (or, once deployed, a PR changes or it's 9am on a weekday)	A run starts
2	Orchestrator	Lists the open PRs through **GitHub MCP**	15 PR numbers and titles
3	Review guide ×15	One fresh subagent per PR reads the title, description and diff through GitHub MCP	A `ReviewBrief` per PR, saved as `/briefs/<pr>.json`
4	Dependency gate	A code tool compares `package.json` on `main` and in each PR and looks up GitHub security advisories; the model judges	A `DependencyReport`, saved as `/deps.json`
5	Code	Computes which PRs edit the same lines, which PR is stacked on another, and each PR's lane	Compact facts
6	Queue planner	Gets the facts in its prompt (no tools) and writes the order, reasons, review-together pairs and collision advice	A `ReviewQueue`, saved as `/queue.json`
7	Code	Checks the queue: every PR exactly once, lanes by rule, blocked PRs and collisions from the facts	The final queue
8	Publish (code)	Writes labels, comments, "Start here" and the pinned issue to GitHub; posts the Slack digest and DMs	Visible results

Example of what it finds (from the seeded demo repo):

PR	What PR Wrangler says	Lane
#8, #9, #12	Tiny, clean, low risk	Quick win
#3	Bug: "Clear completed" keeps the done to-dos and deletes the active ones (`todos.filter((t) => t.done)`)	Fix first
#2	A "restyle" that also removed the delete confirmation, not mentioned in the description	Fix first
#13	Adds `moment` when `date-fns` already does the job, and that version has a security advisory	Fix first
#14	Adds vulnerable `lodash 4.17.15` for a single `debounce` call	Fix first
#11 + #15	Both add `canvas-confetti`, at different versions: review together	Review
#2 / #4, #4 / #5	Rewrite the same CSS lines: merge one first	Collision
#10	Built on #2's branch	Blocked

---

4. Architecture

The agents

Agent	Model (Groq)	Input	Output	Tools it can use
Orchestrator	`openai/gpt-oss-120b`	"Wrangle the queue."	Delegations, then a final summary	`task` (to call subagents), `github_list_pull_requests`, `publish_to_github`, `post_slack_digest`
Review guide	`qwen/qwen3.8-27b`	"Review PR #N."	`ReviewBrief`: summary, kind, risk, start-here line, findings, what to skim	`github_pull_request_read` (GitHub MCP)
Dependency gate	`qwen/qwen3.8-27b`	"Check dependency changes across all open PRs."	`DependencyReport`: per-PR packages, flags (duplicate, vulnerable, version-drift, heavy), explanation, suggestion	`dependency_changes` (code tool), `github_pull_request_read`
Queue planner	`openai/gpt-oss-120b`	"Plan the review queue." plus injected facts	`ReviewQueue`: headline, ordered lanes with reasons, review-together pairs, collisions, blocked PRs	None (facts are injected into its prompt)

How the agents share work

Each subagent runs in its own clean context and saves its full result as a **file in the agent's shared state**:

- `/briefs/<pr>.json`
- `/deps.json`
- `/queue.json`

The orchestrator only gets back a one-line summary, such as *"PR #3: feature, risk high, 1 bug. Saved to /briefs/3.json."* Later steps (the planner, publishing, Slack) read the files. That keeps the orchestrator's context small no matter how many PRs there are.

Where code takes over from the model

Code handles anything with a right answer. The model handles judgment and wording.

Code decides	The model decides
Which packages changed, and which advisories apply	Whether a new package duplicates one the app already has, or is too heavy for its job
Which PRs edit the same lines (checked against `git merge-tree`)	Which PR should win a collision, and how to say it
Each PR's lane (quick win, review, fix first, blocked)	The order within a lane, and a short reason for each PR
That every PR appears exactly once, with only real PR numbers	The headline, and which PRs belong to the same feature
Every write to GitHub and Slack	—

---

5. Tools and technologies

Agent framework

Tool	What it is	How it's used here	Where
LangSmith Managed Deep Agents (MDA)	LangChain's way to build an agent as a folder of files, with LangSmith providing the runtime	The whole agent: `define_deep_agent` with one orchestrator and three subagents; `mda init`, `mda dev`, `mda build`	`agent/agent.py`
Deep Agents harness	The open-source agent loop under MDA: planning, a `task` tool for subagents, a scratch filesystem	Subagent delegation, and the shared state files for results	(built in)

Tool	What it is	How it's used here	Where
LangGraph	The graph runtime underneath Deep Agents	The agent runs as a LangGraph app on a local Agent Server	(built in)
LangChain middleware	Hooks that run around every model call or tool call	Tool scoping, guardrails, saving results, retries, a rate-limit lock	`agent/middleware/`
Structured output (`ToolStrategy`, `ProviderStrategy`)	Forces the model to answer in a fixed schema	`ToolStrategy` for the review guide and dependency gate (they also use tools); `ProviderStrategy` (Groq's native JSON mode) for the tool-less planner	`agent/subagents/`
Pydantic	Python data schemas	`ReviewBrief`, `DependencyReport`, `ReviewQueue`	`agent/subagents/`

Models

Tool	How it's used	Why
Groq (free tier)	Hosts the models	Free and fast
`openai/gpt-oss-120b`	Orchestrator and queue planner	The orchestrator needs to follow a multi-step workflow reliably; the smaller 20b model bundled PRs and skipped steps
`qwen/qwen3.8-27b`	Review guide and dependency gate	Good at reading code and returning structured briefs
`langchain-groq` (`ChatGroq`)	LangChain's Groq client	Retries on rate limits, reasoning-effort and output caps per role

Integrations

Tool	How it's used	Where
GitHub MCP server (remote, `api.githubcopilot.com/mcp/`)	The agents read PRs: list them, then get details, files and patches. Locked to 4 read-only tools, read-only mode and lockdown mode.	`agent/tools/mcp.py`
GitHub REST API	Code tools fetch `package.json` from each branch, security advisories and PR diffs; the publisher writes labels and comments	`agent/tools/dependencies.py`, `queue_facts.py`, `publish.py`
GitHub GraphQL API	Pins the Review Queue issue	`agent/tools/publish.py`

Tool	How it's used	Where
GitHub webhooks	PR events and `/wrangle` comments start a run; the signature is checked with a shared secret	`agent/channels/github.py` (MDA HTTP channel)
Slack Web API (bot token)	Posts the digest to `#daily-pr-review` and DMs authors as the PR Wrangler app	`agent/tools/slack_messages.py`
Slack MCP server	Fallback for sending the same messages; it posts as the installing user, because Slack MCP only accepts user tokens	`agent/tools/mcp.py`, `agent.py`
Slack app	An internal app with trimmed scopes (`chat:write`, `chat:write.public`, `im:write` for the bot)	Slack workspace "PR-Wrangler"

Observability, evaluation and testing

Tool	How it's used	Where
LangSmith Tracing	Every run is recorded: each agent, tool call, input, output, token count, latency and estimated cost. Used to debug token limits, failing models and skipped steps.	Project `pr-wrangler-local`
LangSmith Studio	Watch the agent's graph, chat with it live, inspect the shared state files	`uv run mda dev`
LangSmith datasets + experiments	The 24 planted-answer checks are uploaded as the dataset `pr-wrangler-checks`; an experiment scores a full run: **24/24**	`agent/scripts/langsmith_evals.py`, `checks.py`
Seeded demo repo	`tiny-todo`, a small to-do app with 15 PRs from 7 fictional developers, each hiding a planted problem	`seed/prs.py`, `seed/seed.py`, `github.com/dhanush014/tiny-todo`
Answer key	The right answers, kept outside the repo so the agents can't see them	`seed/expected.json`
Check scripts	Run the agent and compare its output to the answer key (24 output checks, plus "the queue was saved" and "the run didn't error")	`agent/scripts/check_*.py`, `e2e.sh`
Reset script	Wipes everything the agent wrote, so a demo can be re-run from scratch	`seed/reset.py`
git merge-tree	Ground truth for which PR pairs really conflict, used to tune the collision detector	(one-off validation)

Language and tooling

Tool	Used for
Python (async)	All agent code, tools and scripts
uv	Project and dependency management, running the MDA CLI
httpx	Async calls to GitHub and Slack
langgraph-sdk	Test client that talks to the local Agent Server
gh CLI	Creating the demo repo and opening the 15 PRs
Vite + vanilla JS	The demo to-do app

---

6. Guardrails

Group	Guardrail	Why it exists
Access	GitHub MCP is read-only, limited to 4 tools, in lockdown mode	The agents can't approve, merge or change anything, and PR text from strangers is filtered
	Each agent sees only its own tools (`only_tools` middleware)	Subagents automatically inherit every tool; this narrows them, and saves tokens
	PR text is treated as data, never as instructions	Protection against prompt injection through PR descriptions
Order of work	One PR per review task	The orchestrator once crammed all 15 PRs into a single review, far over the token limit
	No planning until every PR is reviewed	The orchestrator once tried to plan after 5 of 15 reviews. Code refused and listed the missing PRs, and the trace shows it obeying.
Output	Typed schemas for every result	Results can be checked and used by code
	Findings whose "fix" is already in the code are dropped	Caught a hallucinated bug on PR #14
	Lanes, blocked PRs and collisions computed by code	The model once dropped whole lists and misfiled a buggy PR
Actions	Never approves or merges; publishing edits its own comments in place	Safe to re-run, and humans keep the decision
	Webhooks must carry a valid GitHub signature	Only GitHub can trigger runs

---

7. Skills demonstrated

Skill	Where it shows up
Multi-agent system design	Splitting the work into an orchestrator plus three subagents by *what context each needs*: one PR in depth, all packages, or a summary of everything
Context engineering	Saving results as state files with one-line summaries back to the orchestrator; injecting precomputed facts into the planner; a per-call prompt cut from 3,557 to 1,952 tokens
Tool and MCP integration	Remote GitHub MCP with header-level restrictions; Slack MCP; finding undocumented behavior by running it (tool naming, MCP tools attaching only to the main agent)
Structured output	Choosing between tool-based and native JSON output per agent, based on a provider limit (Groq can't mix JSON mode with tool calls)
Middleware and guardrails	Code checks that refuse bad steps and tell the model what to do instead
Working within hard limits	Fitting a 15-PR, multi-agent run into Groq's free tier (8,000 tokens per minute, about 200,000 a day per model) by splitting models, capping output, retrying and serializing calls
Deterministic plus LLM hybrid design	Code for facts (collisions validated against git, advisories, lanes, publishing), the model for judgment and wording
Evaluation	Building ground truth first (planted problems and an answer key), graded by code rather than an LLM judge, tracked as LangSmith experiments
Observability-driven debugging	Using traces to find the token bottleneck, the failing model, the bundled tasks and the skipped reviews
Idempotent integrations	Hidden markers so comments are edited, not duplicated; labels replaced, not stacked; a reset script for clean re-runs
Security hygiene	Least-privilege scopes (trimming Slack's auto-added scopes from 30 to 3), webhook signatures, tokens kept in `.env` and out of builds
Product thinking	Putting results where reviewers already work (GitHub, Slack) instead of a new dashboard

8. Problems hit along the way, and the fixes

Problem	Cause	Fix
Each orchestrator call carried about 3,600 tokens, and the briefs piled up in its conversation	Built-in instructions plus every subagent result in its history	Results saved to state files; the orchestrator only sees its own tools
Groq rejected requests outright	8,000 tokens per minute per model; qwen allows only 1,000 output tokens per minute	Models split by role, output capped, thinking turned off on qwen, retries, one review at a time
"JSON mode cannot be combined with tool calling"	A Groq limitation	Tool-based structured output for agents that use tools

Problem	Cause	Fix
A model returned empty or plain-text answers where a tool call was required	Small-model flakiness on Groq	Automatic retries; review guide moved to qwen; planner redesigned with no tools and native JSON
The orchestrator bundled PRs, then skipped them	Unreliable at following a long checklist	Guardrails in code, plus a bigger model
The planner dropped whole lists from its answer	Optional schema fields	Code finalizes the queue from facts
A hallucinated bug on PR #14	Model error	A filter that drops findings whose fix is already in the code
Slack messages showed up under the user's name	Slack MCP only accepts user tokens	Posting switched to a bot token; MCP kept as a fallback
`mda deploy` refused	LangSmith's free plan doesn't include deployment	Runs locally; the webhook and schedule are written, ready for a paid plan

---

9. Results

- **Accuracy:** 24 of 24 output checks pass against the planted answers (26 of 26 including the two run-level checks). Tracked as a LangSmith experiment.
- **Speed:** a full 15-PR run takes about **11 minutes** on Groq's free tier, mostly waiting on rate limits.
- **Cost: $0** on the free tier. At paid Groq prices, LangSmith estimates about **3–4 cents** per full run.
- **Live output:** labels, briefs and "Start here" comments on all 15 PRs, and a pinned Review Queue issue at `github.com/dhanush014/tiny-todo`; the digest and author DMs in Slack `#daily-pr-review`.

---

10. Limitations and next steps

Limitation	Next step
Runs locally; the webhook and 9am schedule need a hosted deployment	`mda deploy` on LangSmith Plus. The code is already written.
Groq's free tier allows only a couple of full runs a day	Groq's paid tier, or a key used only for this project
A full run takes about 11 minutes	A paid tier removes most of the waiting; parallel reviews become possible
"Why" reasons for the review lane are often just restated titles	More specific planner instructions, or code-generated reasons like the fix-first ones
All Slack DMs go to one person, because the team is fictional	Map real teammates in `team.json`
The agent code isn't on GitHub yet	Put `agent/` and `seed/` under version control